# GLM Output Guide

## Overview

GLM produces NetCDF output containing simulated water temperature profiles. Processing this output requires understanding the coordinate system and matching with observations.

## Output File

After running GLM, results are in `output/output.nc`:

| Variable | Description | Shape |
|---|---|---|
| `time` | Hours since simulation start | (n_times,) |
| `z` | Height from lake bottom (not depth!) | (n_times, n_layers, 1, 1) |
| `temp` | Water temperature (°C) | (n_times, n_layers, 1, 1) |

## Reading Output with Python

```python
from netCDF4 import Dataset
import numpy as np
import pandas as pd
from datetime import datetime

nc = Dataset('output/output.nc', 'r')
time = nc.variables['time'][:]
z = nc.variables['z'][:]
temp = nc.variables['temp'][:]
nc.close()
```

## Coordinate Conversion

**Important**: GLM `z` is height from lake bottom, not depth from surface.

```python
# Convert to depth from surface
# Set LAKE_DEPTH based on lake_depth in &init_profiles section of glm3.nml
LAKE_DEPTH = <lake_depth_from_nml>
depth_from_surface = LAKE_DEPTH - z
```

## Complete Output Processing
```python
from netCDF4 import Dataset
import numpy as np
import pandas as pd
from datetime import datetime

def read_glm_output(nc_path, lake_depth):
nc = Dataset(nc_path, 'r')
time = nc.variables['time'][:]
z = nc.variables['z'][:]
temp = nc.variables['temp'][:]
start_date = datetime(2009, 1, 1, 12, 0, 0)

records = []
for t_idx in range(len(time)):
hours = float(time[t_idx])
date = pd.Timestamp(start_date) + pd.Timedelta(hours=hours)
heights = z[t_idx, :, 0, 0]
temps = temp[t_idx, :, 0, 0]

for d_idx in range(len(heights)):
h_val = heights[d_idx]
t_val = temps[d_idx]
if not np.ma.is_masked(h_val) and not np.ma.is_masked(t_val):
depth = lake_depth - float(h_val)
if 0 <= depth <= lake_depth:
records.append({
'datetime': date,
'depth': round(depth),
'temp_sim': float(t_val)
})
nc.close()

df = pd.DataFrame(records)
df = df.groupby(['datetime', 'depth']).agg({'temp_sim': 'mean'}).reset_index()
return df
```

## Reading Observations
```python
def read_observations(obs_path):
df = pd.read_csv(obs_path)
df['datetime'] = pd.to_datetime(df['datetime'])
df['depth'] = df['depth'].round().astype(int)
df = df.rename(columns={'temp': 'temp_obs'})
return df[['datetime', 'depth', 'temp_obs']]
```

## Calculating RMSE
```python
def calculate_rmse(sim_df, obs_df):
merged = pd.merge(obs_df, sim_df, on=['datetime', 'depth'], how='inner')
if len(merged) == 0:
return 999.0
rmse = np.sqrt(np.mean((merged['temp_sim'] - merged['temp_obs'])**2))
return rmse

# Usage: get lake_depth from glm3.nml &init_profiles section
sim_df = read_glm_output('output/output.nc', lake_depth=25)
obs_df = read_observations('field_temp_oxy.csv')
rmse = calculate_rmse(sim_df, obs_df)
print(f"RMSE: {rmse:.2f}C")
```

## Common Issues

| Issue | Cause | Solution |
|-------|-------|----------|
| RMSE very high | Wrong depth conversion | Use `lake_depth - z`, not `z` directly |
| No matched observations | Datetime mismatch | Check datetime format consistency |
| Empty merged dataframe | Depth rounding issues | Round depths to integers |

## Best Practices

- Check `lake_depth` in `&init_profiles` section of `glm3.nml`
- Always convert z to depth from surface before comparing with observations
- Round depths to integers for matching
- Group by datetime and depth to handle duplicate records
- Check number of matched observations after merge